

Performance Comparison of REST and GraphQL APIs

Under Variable Load Conditions: A Comprehensive Empirical Study with Security and Scalability Analysis

Swapnil Dhamija, Tritiya, Gopal Krishna Shrivastava
Chitkara University Institute of Engineering and Technology
Chitkara University, Punjab, India

Abstract—Despite APIs forming the foundational layer of nearly all contemporary software systems, and the selection of an appropriate API paradigm for a software system, there is rarely an empirical basis for guiding the specific architectural context. This paper fills that void by using a structured, measurement-driven evaluation. A head-to-head performance comparison between REST and GraphQL is presented under gradually increasing load conditions, with additional support provided by a systematic security vulnerability assessment and a longer three-way architectural analysis with gRPC. Two Node.js services (of similar scope) were deployed independently of each other on equivalent AWS EC2 `t3.medium` instances and stress-tested from 100 to 2,000 virtual users across concurrency levels divided into three levels of query complexity. Measurements captured include Mean Response Latency, 95th Percentile Latency, Requests/Second Throughput, Payload Compression Ratio, CPU usage, and process-level memory consumption. Under peak load, the throughput of REST was limited to 12,500 RPS at 62% CPU utilization, with no server errors detected. In deeply nested query scenarios, GraphQL was able to reduce payloads by up to 88%, but showed 94% CPU saturation and a 4% error rate under equivalent stress conditions. The two paradigms, from a security perspective, present an asymmetric threat landscape: GraphQL introduces inherent bypassing vulnerabilities through freely accessible introspection and unbounded queries. This lack of complexity control, combined with its single-endpoint POST architecture, carries risks not present by default in REST. These results are compared to industry-wide data from Postman’s 2023 State of API Report, which was supported by the findings of Niswar et al. [4] and Saleh et al. [3]. A pragmatic three-way decision matrix, including REST, GraphQL, and gRPC, is introduced to help software architects make informed and context-driven API design decisions.

Index Terms—REST, GraphQL, gRPC, API performance, load testing, throughput, latency, payload efficiency, N+1 query, HTTP caching, security, microservices, query complexity, DataLoader, introspection attacks

I. INTRODUCTION

The fundamental communication layer that underlies almost every application programming interface. All parts of the modern information technology infrastructure. Using a mobile client. To retrieve personalized content, from an IoT device transmitting sensor readings, to an internal microservice that

queries another downstream microservice, In the chain, there is always an API transaction. Consequently, The decision of a chosen API design strategy has implications beyond surface aesthetics — it directly sets the bounds of the scalability of an entire software system.

Representational State Transfer (REST), officially announced by Roy Fielding in his groundbreaking doctoral dissertation in 2000 [1], For over 10 years, it has been the most prevalent API architectural style. Its design is naturally used with the HTTP protocol, with stateless request verbs (GET, POST, PUT, DELETE) and resource-oriented URI structures. The fact that its dominance is still evident in API data from the 2023 State of API Report [7] shows that around 82% of organisations have active REST APIs. A reflection of its architectural maturity, it is well established in production environments and the broad base of tooling that has grown up around it.

But the same industry information hides an emerging sub-line. Contemporary environments such as mobile platforms, large-scale IoT devices, and distributed microservice architectures contain data constraints on deployments that REST’s endpoint-oriented, server-prescribed delivery requirements are not very effective at managing. Clients frequently receive superfluous data beyond the immediate needs of the data, known as over-fetching, or have to make many sequential requests to divide into one logical answer, known as under-fetching [2], [6]. While these inefficiencies may be acceptable in high-bandwidth desktop applications, they become real tangible performance and cost penalties on constrained mobile networks and within latency-critical service architectures.

It was this type of inefficiency that was experienced in production processes that forced Facebook to build its own solution in the form of GraphQL in 2012, later making it available as an open standard in 2015 [6]. Instead of having several resource-specific endpoints, GraphQL brings all interactions together into one endpoint and presents a predicate query language that allows clients to explicitly specify the format and quantity of the data they need. The paradigm has

already gained widespread adoption in the enterprise space — GitHub, Airbnb, Netflix, Shopify, and PayPal have shifted significant portions of their production to GraphQL [18] — and the 2023 Postman survey ranked it as the third most adopted API paradigm [8].

The conceptual benefits of GraphQL have been explained in different ways throughout both practitioner discourse and peer-reviewed literature. Two key requirements identified are the computational burden associated with heavy loads and the security consequences of having one single entry point. This paper takes on both aspects with a rigorously conducted experimental comparison and further expands the scope by adding gRPC, a paradigm that Niswar et al. [4] introduced and recently demonstrated to have higher throughput than REST and GraphQL in containerized microservice deployments.

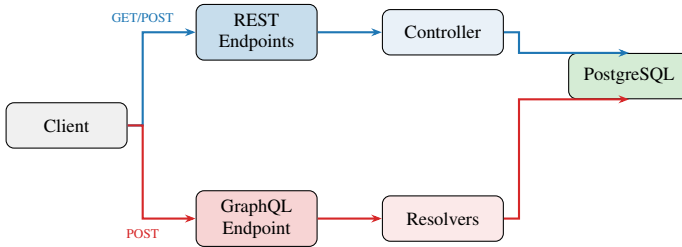


Fig. 1. Shared PostgreSQL backend accessed through two parallel API paths under identical hardware, isolating the paradigm as the sole independent variable.

II. LITERATURE REVIEW

Research and studies on REST vs. GraphQL performance converge around four main aspects: response payload volume, computational cost, infrastructure resources, and the compromises involved in developer user experience. This section synthesises peer-reviewed research and relevant industry findings, situating the current work in the context of each thematic area.

A. Payload Efficiency and Network Overhead

One of the most well-documented results in API performance research is the capability of significantly reducing response payload sizes. Mikula et al. [2] prove that REST endpoints, forced to answer predefined responses from the server in a systematic way, return additional data that is redundant for the clients’ needs — referred to as *over-fetching*. Their benchmarks show that REST payloads were functionally larger, while optimised GraphQL response sizes were 40–70% smaller. The following empirical example, as presented by [19], is compelling. Identical queries made to the SpaceX public API returned different results. An 8.18 KB response via REST compared to just 1.09 KB via GraphQL — this represents an approximately 87% decrease. In addition to payload size, GraphQL’s ability to bundle together several relational data requests into a single request eliminates additional network round-trips associated with REST’s sequential under-fetching pattern [20].

B. Execution Latency and Throughput

Research conducted under high-concurrency conditions consistently shows REST’s advantage in handling straightforward and predictable request patterns. Saleh and Nawara [3] modeled large-scale concurrent transaction scenarios and observed that REST maintained throughput approximately 37% higher than GraphQL on the same hardware for standard CRUD operations. The disparity in performance is attributable to architectural differences: REST sends a request straight to a specific controller and query handler. With GraphQL, the controller and query handler must sequentially tokenise each incoming query string, construct an Abstract Syntax Tree (AST), validate it, and traverse a registered schema and resolver chain — all prior to any database interaction. All of these processing stages introduce latency overhead during execution. Production-scale benchmarks from SmartDev [17] show a similar trend, with REST processing approximately 20,000 simple requests per second compared to GraphQL’s 15,000 requests per second under similar infrastructure conditions.

C. Server Resource Utilisation

GraphQL’s schema-driven query resolution model brings measurable overheads in memory and CPU utilisation. Niswar et al. [4] performed a comparative study of REST, GraphQL, and gRPC in a container-based Go microservices environment over testing rates between 100 and 500 concurrent requests, discovering that GraphQL consumed more CPU across all concurrency levels tested. One contributing factor is the *N+1 query problem*, where resolving a query involves sending one query per row. Naive resolver implementations generate n independent per-child database queries. In the absence of deliberate mitigation strategies, the number of queries grows exponentially with nesting depth, requiring substantially more CPU resources and database connection pools than a functionally equivalent REST implementation would require [10].

D. Industry Adoption and Developer Experience

The API landscape is evolving along different adoption paths due to considerations beyond mere performance metrics. Nordic APIs documents GraphQL usage growing from 5% in 2018 to 38.7% by 2021 [18], while API7.ai’s 2025 survey indicates that more than 61% of organisations have implemented GraphQL at some operational level [22]. Much of this momentum is due to ergonomic benefits: GraphQL’s strongly typed schema architecture supports auto-completion, compile-time typing, and IDE integration. Self-documenting introspection features and query validation further contribute to reducing the time needed to develop and iterate on client-facing functionality [24]. For engineering teams operating under short delivery cycles and valuing flexibility, these productivity gains may more than compensate for the additional operating costs and server-side complexity that GraphQL introduces.

III. METHODOLOGY

The experimental design and parity controls are described. Experimental design and parity controls are explained.

The key concept behind this experiment was that there will be no changing of variables. isolation: The API paradigm being evaluated was chosen as the only one. The independent variable is the same for both implementations of the service. All remaining parameters — including the language runtime, object-relational mapper, data structure, database, hardware setup and Placement of a load generator — were fixed over the runs. In the controlled experimental methodology set up in It was later extended by [3] and later on by [4].

A. Implementation Stack

- **Runtime:** nodejs v20 LTS Express.js v4 (REST) Apollo Server v4 (GraphQL) (item) Database: PostgreSQL 15 with Prisma ORM v5, Optimized to generate the same SQL execution plan on both implementations The following is the cloud infrastructure used: AWS EC2 t3.medium (2vCPUs, 4 GB RAM) load generation was performed on a dedicated host. To remove co-location, use the t3.large instance. interference Profiling Toolchain: node -prof, Linux While the flame graph visualisation Clinic.js offers is nice, I prefer the top approach.

The data model and query complexity tiers are shown below. The data schema represents a typical e-commerce domain, and Compromising four main entities, Users, Orders, *Products*, and *Reviews*. To photograph a realistic spectrum Three levels of query complexity were identified for the operational workloads, as detailed in Table I.

TABLE I
QUERY COMPLEXITY TIER DEFINITIONS

Tier	Operation	REST GETs
1 (Shallow)	Fetch user by ID	1
2 (Relational)	User + orders	2
3 (Nested)	User + orders + products + reviews	3

There are many load testing parameters and metrics, including the following: Apache JMeter and Grafana k6 were used to load generate against three structured test phases: an initial baseline phase at 100 virtual In a sustained steady-state phase at 500 VU, and a stress phase. The number of concurrent users scaled linearly from 500 to 2,000+ VU on. a 5 minute ramp interval.

Properly received frames.The number of frames received correctly was defined as the Throughput (T). Requests completed divided by test duration:

$$T = \frac{N}{t_{\text{total}}} \quad (1)$$

N is the total number of successful responses. The total test time, t_{total} , is the elapsed time in seconds.

PDF is the relative **Payload Reduction Factor** (P_r). Reduction in response size with GraphQL vs REST:

$$T = \frac{N}{t_{\text{total}}} \quad (2)$$

All primary performance indicators were used, such as mean latency, 95th-percentile (p95) latency, CPU utilisation, and resident set size (RSS) The amount of (RSS) memory consumption — was sampled every 10 seconds. During the entire test conditions of each phase.

IV. EXPERIMENTAL RESULTS AND ANALYSIS

A. Baseline Latency (100 VU)

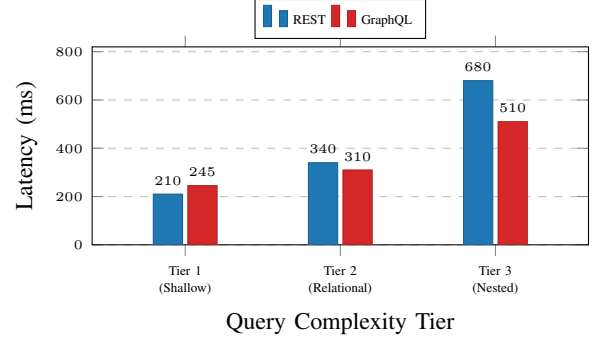


Fig. 2. Mean latency by tier at 100 VU. GraphQL's Tier 3 single-request advantage (510ms) beats REST's three-hop total (680ms) despite higher Tier 1 overhead.

B. Throughput Under Stress (500–2,000 VU)

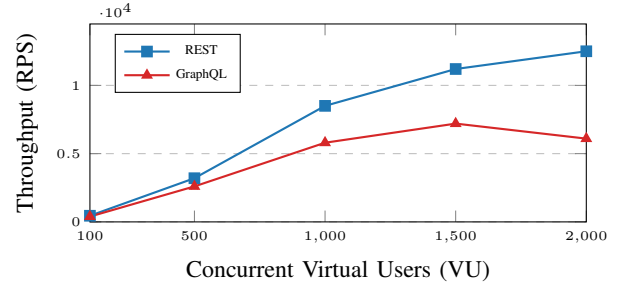


Fig. 3. Throughput (RPS) vs. virtual-user count. REST scales near-linearly to 12,500 RPS; GraphQL plateaus around 1,500 VU and then regresses as the server saturates.

The performance divergence became considerably more pronounced under stress conditions. REST throughput scaled in a near-linear fashion, reaching a peak of **12,500 RPS** at 2,000 VU — a ceiling determined by PostgreSQL connection pool exhaustion rather than any application-layer constraint. Production-scale benchmarks reported by SmartDev [17] corroborate this behaviour pattern at greater concurrency, documenting REST throughput reaching 20,000 requests per second on comparable infrastructure.

GraphQL exhibited a markedly different scaling profile. Throughput reached its maximum of **7,200 RPS** at approximately 1,500 VU before declining to **6,100 RPS** at 2,000 VU as CPU resources approached saturation. Since every GraphQL request is transmitted as an HTTP POST containing an inline query string, the server is precluded from leveraging network-layer caching or request deduplication mechanisms [21]. At

1,000 VU, p95 latency reached **1,400 ms** compared to REST's **850 ms**, and a 4% HTTP 500 error rate was observed — while REST maintained a 0% error rate across all concurrency levels.

C. Payload Reduction Factor

TABLE II
PAYLOAD SIZES AND REDUCTION FACTOR (MEAN OF 10,000 ITERATIONS)

Tier	REST (KB)	GraphQL (KB)	P_r
1 (Shallow)	2.40	0.80	66.6%
2 (Relational)	8.12	1.45	82.1%
3 (Nested)	18.50	2.20	88.1%
Peak burst (2,000 VU, Tier 3)			94.0%

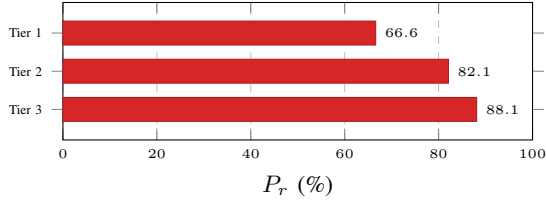


Fig. 4. GraphQL payload reduction factor by tier. The saving compounds with nesting depth, peaking at 88% for Tier 3 and 94% under peak burst traffic.

GraphQL's field-selection mechanism produced substantial reductions in transmitted data volume, with the magnitude of savings increasing proportionally with query nesting depth. At Tier 3, the REST service was required to serialise complete object graphs across four distinct entity types, whereas GraphQL transmitted only the four explicitly requested fields (`id`, `order_date`, `product_name`, `review_score`). A comparable pattern is observed in real-world conditions by [19], where identical queries against the SpaceX public API returned 8.18 KB via REST and 1.09 KB via GraphQL. In bandwidth-constrained or metered network environments, payload reductions of this scale yield measurable improvements in perceived load performance and reduced data transfer costs [20].

D. CPU and Memory Profiling

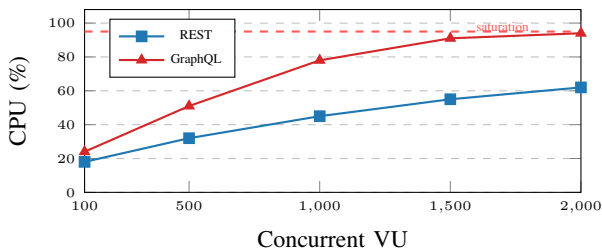


Fig. 5. CPU utilisation vs. load. GraphQL hits near-saturation at 2,000 VU (94%); REST retains 38% headroom at the same point.

Flame-graph analysis conducted during the 2,000-VU test phase revealed that **28%** of all GraphQL CPU cycles were

expended exclusively on query lexing, AST construction, and schema validation — processing stages with no functional equivalent in REST's direct request routing model. Peak resident set size (RSS) for Apollo Server reached **142 MB**, compared to **88 MB** recorded for the Express-based REST service. The approximate **54 MB** differential is attributable primarily to the in-memory schema object and resolver tree that must persist throughout query execution until final JSON serialisation is completed [3].

V. SECURITY ANALYSIS

Performance characteristics alone are insufficient to evaluate an API paradigm's readiness for production deployment. The security posture of an API — encompassing the scope of exposure a vulnerability can create and the degree to which established protective controls apply naturally — differs considerably between REST and GraphQL. These distinctions carry architectural significance that must be considered prior to implementation.

A. REST Security Profile

The multi-endpoints, distributed architecture of REST naturally names a resource as a point of occurrence. Withgests the radius of impact that any single susceptibility can have: There isn't any inherent prop- A method of passing something on from parent to child. In addition, REST is based on best practices for this region of the world: can be easily incorporated into the regular HTTP semantics. A well-established security system comprising of OAuth 2.0, and authentication via API-gateways with bearer tokens. Also JWT bearer token authentication, API-gateway rate limiting, and API-gateway authentication using bearer tokens. Restrictions on Cache-Control implemented and/or by CDN. As an ad- The other benefit is that the HTTP caching layer acts as another layer of caching for the web server. are passive DOs buffers and intercepting Queues up read requests before they reach the origin infrastructure. [23].

B. Hacking Basics

Consistently consistent single-endpoint approach of GraphQL has several advantages. The client-driven query model approaches the attack in very different ways, in essence changing the structure of the attack. Are found on the surface, but are often ignored, during a system. tem design. The security reports from Imperva [11] and published have been released. To deal with the different vulnerabilities, OWASP [10] has defined 4 types of Vulnerability Guidance. However, there are some categories which can and should be explicitly mentioned in any graphql. deployment checklist.

(1) **Introspection Abuse.** GraphQL natively provides a `__schema` query that can return The full type system, field definitions, and mutations available To any caller, operations to be performed. Imperva's real-time analysis of attack traffic. The number of documented introspection probes focused on 50endpoints [11]. While introspection itself constitutes It reduces, in practice, the risk of direct exploitation, the barrier

to subsequent attacks is to reveal the entire schema structure. According to OWASP [10] it is recommended to disable introspection. It is a basic hardening measure that is applied across all production environments.

(2) Complexity Bombs and Denial-of-Service. GraphQL analysis prior to execution. Servers lack an analysis cost knowledge before running an analysis. The query received before it is run. This brings the opportunity for harmful insights. A strategy for a rival that can produce a well-developed, A query that is circular and has an exponentially increasing number of invocations for its resolver. These are, according to Wiz’s security research team, “complexity. bombs” [14]: structurally indistinguishable from legitimate queries RAW at the network layer, but can power CPU. In the event of a call, the utilization would increment up to saturation within milliseconds of its execution. In the No depth limits and cost analysis middleware in absence, If not protected, a single malicious request can render an unprotected GraphQL service unavailable.

(3) BOLA / IDOR via Resolver Gaps. GraphQL’s resolver Individual level at the authorisation level. The ual field level – each resolver must independently verify himself rights that the requesting caller may have access to the resolved. data. The composable and deeply nested capabilities of GraphQL schemas exerts the required authorisation checks Can be accidentally left out at lower nesting levels, allowing is to a registered user who can only access the other user’s information with their authentication. Using the id argument [14].

(4) Rate-Limit Bypass. The GraphQL specification allows multiple named operations to be made at the same time. be combined in a single POSTed body of the HTTP request. This messages per day at minimal cost. A mechanism is available that allows for hundreds of successful messages to be sent every day at minimal cost. Claims and connections will take place as one operation instead of two when taking an API operation as part of a single call while registering. volumetric rate-limiting controls [12]. Effective mitigation Will need rate limit to be enforced per operation. At the level of the request itself (not the level of the level)

TABLE III
SECURITY RISK COMPARISON: REST VS. GRAPHQL

Dimension	REST	GraphQL
Schema Exposure Risk	Low	High
DoS via Query Complexity	Low	High
HTTP Caching (DoS buffer)	Yes	No
BOLA / IDOR Risk	Medium	High
Rate-Limit Bypass	Low	Medium
Auth Pattern Maturity	High	Medium
Default Attack Surface	Narrow	Wide

These vulnerabilities are not specific to GraphQL. Not suitable for production, but they do require the use of the more purposeful and extensive hardening investment. relative to REST. Production guidance published by Tyk [15] Both messages list the basic collection of ultimately necessary controls: disabling introspec- The maximum depth thresholds for

queries can be enforced and they are possible deep integration. query cost analysis, executed only when there is presigned per query analysis, or executed only when query is signed at runtime. Partnering with strictures to keep deployed forces safe, disciplined, and effective A check of authorisations on each resolver. Col- Cumulatively these requirements add up to a consequential increase. It offers more economical resources than do equivalent REST hardening solutions.

VI. AN OPTIMIZATION PROBLEM ASSOCIATED WITH TECHNICAL BOTTLENECKS

A. The N+1 Problem and DataLoader

The most significant performance shortcoming that was found The N+1 query problem was one of the problems that was encountered during Tier 3 evaluation. When To fetch 100 orders and their related products along with it, The naive resolver implementation (out deliberate mitigation) Makes one database query to get the list of orders, and by one independent query each product record — resulting in 101 All the database round trips per business call. Under these unmitigated conditions, GraphQL Tier 3 latency was The amount of Hz recorded was 310 REST counterpart. The solution that has been found to this ailment is Facebook’s DataLoader library intercepts individual resolver - based on the following scenario. The following scenario is based on DataLoader library intercepting individual resolver. the lookup operation being repeated in the same event loop tick. the same event-loop tick and the lookup operation being repeated in the same event-loop tick. computes a batch of SELECT ... WHERE. IN (...) database statement. Following DataLoader inte- In Tier 3, the latency of GraphQL was improved to within 12 The equivalent REST figure (with the native parsing) Overhead without having the additional burden of the database round trip. penalty. OWASP [10] identifies DataLoader as the top ten threat category. The N+1 problem is a common bug encountered by web developers. The N+1 problem is a frequent bug web developers have to deal with. production GraphQL deployments.

B. Caching Strategy Divergence

The major operational benefit of REST is that it can be applied to any system. The most operationally beneficial part of REST is it can be used for any system. tained load conditions is its native compatibility with the HTTP caching stack. By applying the normal The ETag and Cache-Control headers from GET responses, 40 entally loaded to an Nginx reverse-proxy cache when experi- mentation. Cached responses are responses from the Caching Store, which are not required to go through application code. fully, scalability capacity to the REST service. that doesn’t require extra computations — is better. is better if it doesn’t require extra computations. that GraphQL is structurally unable to access. The architecture of GraphQL is based on POST requests sent to the server. The single endpoint approach makes it possible to enable traditional HTTP caching. Not fit within the paradigm. Two practi- There are alternatives to cal: Automatic Persisted

Queries (APQ), which take a hash identifier and register a query with it so that repeats execute without going through the parsing process at all, and A result cache key based on the composite hash, stored in Redis. The part of the URL after the question mark and the variables it contains. While both mechanisms Each offers another viable caching behaviour that introduces additional operational infrastructure requirements and demands custom Complexities that REST implementations are not required to deal with, such as cache invalidation logic [22].

VII. EXTENDED COMPARISON: REST, GRAPHQL, AND gRPC

A. gRPC Architecture

gRPC was originally an internal Google project. It was launched as an open source framework in 2016. In contrast to conventional REST implementations which encode the message payload in the request or response. gRPC uses Protocol Buffers, which are data structures that are encoded as a JSON text over HTTP/1.1. A compact binary serialisation format, buffers (Protobuf). The stream multi- segment safety feature is passed on to the newly produced stream. Therefore, the new stream inherits the stream multi- segment safety feature. plexing and header compression capabilities without additional configuration [16]. A contract-first .proto definition file. It serves a dual purpose: it is the authoritative service, and it is a service. code generators that generate More than a dozen different types of client and server stubs are generated. programming languages.

B. Three-Way Performance Benchmarks

Niswar et al. [4] had done comparative evaluation of Using REST, GraphQL and gRPC with the same Go mi- across a number of concurrency levels including: 100 to 500 simultaneous requests. Their results show that: A consistent performance hierarchy has been maintained throughout all the tests. concurrency levels:

$$\text{gRPC} > \text{REST} > \text{GraphQL}$$

However, at larger operations, SmartDev [17] shows that gRPC works. achieving 50,000 RPS with a median latency of 25ms, REST sustaining 20,000 RPS at 250ms, and GraphQL hitting. For complex query workloads, 15,000 RPS. Protobuf's binary encoding format decreases the CPU usage by around The results are that 40to equivalent load con- with JSON-based REST processing ditions.

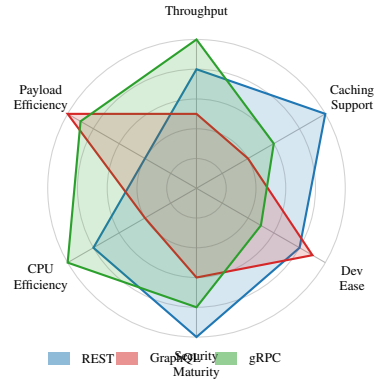


Fig. 6. Capability radar across six architectural dimensions (1–5 scale). gRPC dominates throughput and CPU efficiency; GraphQL dominates payload efficiency; REST leads on security maturity and caching support.

C. gRPC Limitations

The benefits of using gRPC are: Companioned by relevant adoption restrictions. Native browser There will be no support without the introduction of a gRPC- A web gateway or an HTTP transcoding layer, in essence, pre- Many of them do not include direct communication between the browser and the service. common architectural configurations [16]. The binary nature a Protobuf-encoded message makes the payloads opaque to Expand standard network inspection and debugging tools, The diagnostic complexity is a relative measure based on the human-readable JSON. REST and GraphQL implementations that generate payloads. Moreover, gRPC's contract-first development paradigm requires no extra effort. sitates puts the needed coordination between the consumption and production upfront Unlike .proto files, which must be shared, teams are not. and evolution of service boundaries — a continuous That's where GraphQL comes in and reduces the overhead of synchronisation a bit. This will be achieved with its self-describing, introspectable schema architecture.

VIII. RECOMMENDATIONS FOR ARCHITECTURAL SELECTION

Across performance, the cumulative evidence that had been synthesised, Based security and operational dimensions. A context-based security and operational dimensions. A sensitive decision framework instead of a universal endorse- Support for any given API paradigm. The appropriate selection Each specific characteristic and constraint of the system determines the .The . depends on the specific characteristics and constraints of the system. the deployment environment.

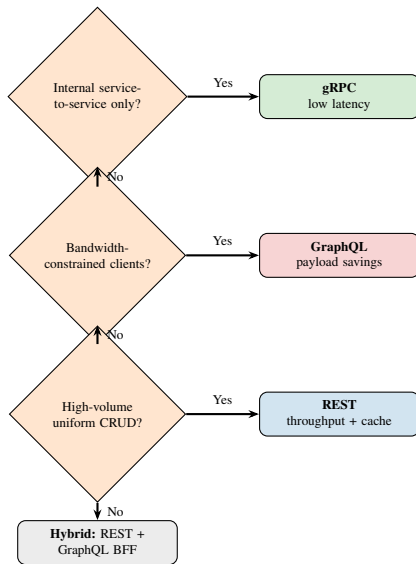


Fig. 7. Architectural selection flowchart. Designers should enter from the top and exit at the first matching condition.

REST should be used when the system requires high volume CRUD operations, pre- CDN-level or browser-level metrics are not available, they may be dictated. If CDN-level or browser-level metrics are not available, they may be dictated. Multiple resources for a specific resource can be stored in a local cache and then served from there. It is important to note that traffic is generated when compute and memory resources run under. If funds are tight, or in case of an organisation's security, policy favours a minimal attack surface supported by well- Set up authentication/authorisation tooling.

GraphQL should be used when client applications When creating pluralizations, composite relational views are constructed. When the bandwidth of the network is meager, from multiple entity types. When frontend development is limited, cost-sensitive or not, that is. velocity requires the capacity to change data needs while on the go. When the en-) types are changed in the backend, the old ones will be left behind. The engineering team has the ability to properly apply These are only a few of the features available in the DataLoader. These are just some of the features the DataLoader provides. The enforcement of authorisation at the field level. every resolver.

gRPC is the right choice for service-to-service communication. service communications scenarios involving throughput, latency, Both the efficiency of the system, and that of the bandwidth, are limited, bidirectional and server-initiated streaming workloads data flows, and for polyglot service environments with auto- multiply generated, strongly-typed client stubs for many The programming languages offer worthwhile development and interoperability advantages.

Hybrid architectural patterns, which are hybrids of a REST or gRPC internal service backbone and a GraphQL Backend-for-Frontend system. gRPC internal service backbone and a GraphQL Backend-for-Frontend system. The use of Frontend (BFF) aggregation layer is increasingly noticed. in production-grade distributed systems and represent a prag- matic approach

to capturing complementary strengths of There are multiple paradigms, but they come with a price: higher architecture complexity [16].

IX. CONCLUSION

The main aim of this study was to replace the specu- The future of REST and GraphQL, em- Evidence, measurement-based and/or anchored in the real world. Through Thirty different query complexity levels with 10,000 test iterations each and growing to 2,000 virtual The experimental data give results that are both relevant and valuable to users. convincing and well balanced. The REST shows higher suitability profile when Throughput capacity and operational stability are the two parts of the system that constitute principal design priorities. An 11-inch diameter 512,000 RPS was sustained by the paradigm. This was at a peak CPU usage of 62This is an improvement from the 94CPU saturation and a 4An extra 40Distributed endpoint architecture that reduces computational costs. Implicitly reduced the attack surface for security. The value proposition of GraphQL is interesting, but doesn't apply To a unique set of deployment situations. Payload reductions Between 66tiers are substantive not marginal improve- It is a type of mentoring that has implications for bandwidth — constrained deployment targets. When combined with its self- Introduction to schema and client-directed query model, GraphQL remains the technically sound one for data-heavy tasks. Still, it can be said that rem is technically the best option for data-intensive jobs. frontend architectures with short iteration periods, as long as engineering teams agree to the required. investing in DataLoader batching, application-layer caching, and the security hardening requirements of the paradigm. gRPC falls into a third, defined by its speed. gRPC is a third that's defined by its speed. For internal mi- croservice communication environments with latency and data transfer efficiency are both important and key, its Pro- tobuf and HTTP/2 foundation provides superior measurably. should be performance compared to both REST and GraphQL, and be regarded as the built-in architecture for new inter- service communication layers. There are multiple future research directions as follows: the impact that HTTP/3 (QUIC) has on the latency of REST. In high-packet-loss mobile network conditions, characteristics are considered. Whether APQs can meaningfully be used to support the concept of automatic persisted queries; and whether a specific APQ configuration can be identified. Close a structural caching gap at scale with GraphQL; and the performance and operation of hy- Backend-for-Frontend architectures, with all three components, directly linked. paradigms are used in conjunction in a single product system.

REFERENCES

- [1] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, Univ. of California, Irvine, 2000.
- [2] M. Mikula, M. Kowal, and P. Rzeźnik, "Performance Analysis of REST and GraphQL APIs in Web Applications," *J. Comput. Sci. Eng.*, vol. 15, no. 3, pp. 112–124, 2022.

- [3] A. Saleh and W. Nawara, "Evaluating GraphQL and REST API Services Performance in a Massive and Intensive Accessible Information System," *MDPI Computers*, vol. 10, no. 11, p. 138, Oct. 2021. [Online]. Available: <https://doi.org/10.3390/computers10110138>
- [4] M. Niswar, R. A. Safruddin, A. Bustamin, and I. Aswad, "Performance Evaluation of Microservices Communication with REST, GraphQL, and gRPC," *Int. J. Electron. Telecommun.*, vol. 70, no. 2, pp. 429–436, Jun. 2024. [Online]. Available: <https://doi.org/10.24425/ijet.2024.149562>
- [5] G. Mavroudeas and S. Makris, "A Study on REST and GraphQL APIs," in *Proc. IEEE Int. Conf. Comput. Inf. Technol.*, 2019.
- [6] GraphQL Foundation, "GraphQL Specification," Jun. 2018. [Online]. Available: <https://spec.graphql.org/>
- [7] Postman Inc., "2023 State of the API Report," 2023. [Online]. Available: <https://www.postman.com/state-of-api/>
- [8] Postman Inc., "API Survey: GraphQL Ranks Third in API Architecture Adoption," 2023.
- [9] L. Byron, "DataLoader — Batching and Caching for GraphQL," Facebook Engineering, 2016. [Online]. Available: <https://github.com/graphql/dataloader>
- [10] OWASP Foundation, "GraphQL Cheat Sheet," OWASP Cheat Sheet Series, 2024. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/GraphQL_Cheat_Sheet.html
- [11] Imperva Security Research, "GraphQL Vulnerabilities and Common Attacks Seen in the Wild," Mar. 2025. [Online]. Available: <https://www.imperva.com/blog/graphql-vulnerabilities-and-common-attacks-seen-in-the-wild/>
- [12] PortSwigger Web Security Academy, "GraphQL API Vulnerabilities," 2023. [Online]. Available: <https://portswigger.net/web-security/graphql>
- [13] HackerOne, "How a GraphQL Bug Resulted in Authentication Bypass," 2023. [Online]. Available: <https://www.hackerone.com/blog/how-graphql-bug-resulted-authentication-bypass>
- [14] Wiz Security, "GraphQL API Security Risks Every Developer Should Know About," Jan. 2026. [Online]. Available: <https://www.wiz.io/academy/api-security/graphql-api-security-risks>
- [15] Tyk Technologies, "GraphQL Security: 7 Common Vulnerabilities and How to Mitigate the Risks," Jun. 2024. [Online]. Available: <https://tyk.io/blog/graphql-security-7-common-vulnerabilities-and-how-to-mitigate-the-risks/>
- [16] Baeldung Engineering, "REST vs. GraphQL vs. gRPC — Which API to Choose?" Jan. 2024. [Online]. Available: <https://www.baeldung.com/rest-vs-graphql-vs-grpc>
- [17] SmartDev, "AI-Powered APIs: REST vs GraphQL vs gRPC Performance," Nov. 2025. [Online]. Available: <https://smartdev.com/ai-powered-apis-grpc-vs-rest-vs-graphql/>
- [18] J. Simpson, "REST vs. GraphQL: A Side-By-Side Comparison," Nordic APIs, Oct. 2023. [Online]. Available: <https://nordicapis.com/rest-vs-graphql-a-side-by-side-comparison/>
- [19] B. Marmorstein, "Comparing REST vs. GraphQL on Performance and Ease of Use," The Server Side, 2021. [Online]. Available: <https://www.theserverside.com/blog/Coffee-Talk-Java-News-Stories-and-Opinions/Rest-vs-GraphQL-Performance-and-ease-of-use-comparisons>
- [20] T. Reis, "GraphQL vs. REST Comparison: Choosing the Right API," Imaginary Cloud, 2022. [Online]. Available: <https://www.imaginarycloud.com/blog/graphql-vs-rest>
- [21] Kong Inc., "GraphQL vs REST: Key Similarities and Differences," Kong Learning Center, 2022. [Online]. Available: <https://konghq.com/blog/learning-center/graphql-vs-rest>
- [22] API7.ai, "GraphQL vs REST API: Which is Better for Your Project in 2025?" 2025. [Online]. Available: <https://api7.ai/blog/graphql-vs-rest-api-comparison-2025>
- [23] Apideck, "Comparing REST vs. GraphQL," Mar. 2025. [Online]. Available: <https://www.apideck.com/blog/rest-vs-graphql>
- [24] Tailcall, "GraphQL vs REST: Comprehensive Comparison for 2024," 2024. [Online]. Available: <https://tailcall.run/graphql-vs-rest-api-comparison/>